



SemiDAW
Travel Agency

Tecnologías: HTML5, CSS, JavaScript, Java, Spring Boot, PostgreSQL

Metodología: GitFlow

Curso académico: 2025-2026

Contenido

ÍNDICE	2
1. Introducción	3
2. Objetivos del proyecto	3
3. Arquitectura general	3
4. Diseño de la Base de Datos	4
5. Arquitectura en capas (Java)	7
6. Frontend (HTML, CSS, JS)	9
7. API REST	10
8. Metodología GitFlow	11
9. Planificación (Diagrama de Gantt)	11
10. Conclusiones	13

ÍNDICE

- 1. Introducción**
- 2. Objetivos del proyecto**
- 3. Arquitectura general**
- 4. Diseño de la base de datos**
 - 4.1 Modelo Entidad–Relación
 - 4.2 Modelo Relacional
 - 4.3 Modelo Físico
- 5. Arquitectura en capas (Java)**
 - 5.1 Entities
 - 5.2 Repositories
 - 5.3 Services
 - 5.4 Controllers REST
 - 5.5 DTO + Mapper
- 6. Frontend (HTML, CSS, JS)**
- 7. API REST**
- 8. Metodología GitFlow**
- 9. Planificación (Diagrama de Gantt)**
- 10. Conclusiones**

1. Introducción

Este proyecto desarrolla una aplicación web completa para una agencia de viajes ficticia, integrando:

- Interfaz web con HTML5, CSS y JavaScript
- Backend en Java con arquitectura en capas
- API REST para operaciones CRUD
- Base de datos relacional
- Metodología GitFlow

El objetivo es simular el funcionamiento de una aplicación real sin necesidad de despliegue en producción.

2. Objetivos del proyecto

Los objetivos del proyecto son los siguientes:

- Aplicar el diseño y desarrollo web implementando una interfaz web funcional y clara utilizando HTML5, CSS y JavaScript.
- Diseñar y gestionar una base de datos modelando la información mediante diagramas Entidad-Relación y modelo relacional, respetando las relaciones entre entidades.
- Desarrollar la lógica de una arquitectura backend implementando una arquitectura en capas en JAVA, creado y gestionando endpoints, REST para operaciones CRUD.
- Integrar frontend y backend conectando la interfaz web con los servicios backend para la gestión de datos sin un despliegue real.
- Utilizar herramientas de desarrollo profesional aplicando control de versiones mediante Git y GitHub siguiendo la metodología estructurada GitFlow.
- Gestionar el proyecto mediante diagramas de Gantt, planificando las tareas y control de desviaciones.
- Fomentar el trabajo en equipo estableciendo roles definidos, coordinando las tareas y responsabilidades dentro del grupo.
- Desarrollar la documentación técnica elaborando una documentación clara y estructurada del proyecto.
- Mejorar la resolución de problemas identificando y solucionando las incidencias surgidas durante el desarrollo del proyecto.

3. Arquitectura general

La arquitectura sigue un modelo cliente–servidor:

- Frontend: HTML5, CSS, JavaScript
- Backend: Java + Spring Boot

- Comunicación: API REST (JSON)
- Base de datos: PostgreSQL
- Flujo general:
- Frontend → API REST → Servicios → Repositorios → Base de Datos

4. Diseño de la Base de Datos

Se diseña una base de datos que almacenará los datos de usuarios, destinos, guías y pasaportes de la agencia de viajes SemiDAW Travel Agency., A continuación se implementa el esquema Entidad-Relación, el esquema Relacional y el esquema físico.

4.1 Esquema Entidad-Relación (ER)

Entidades:

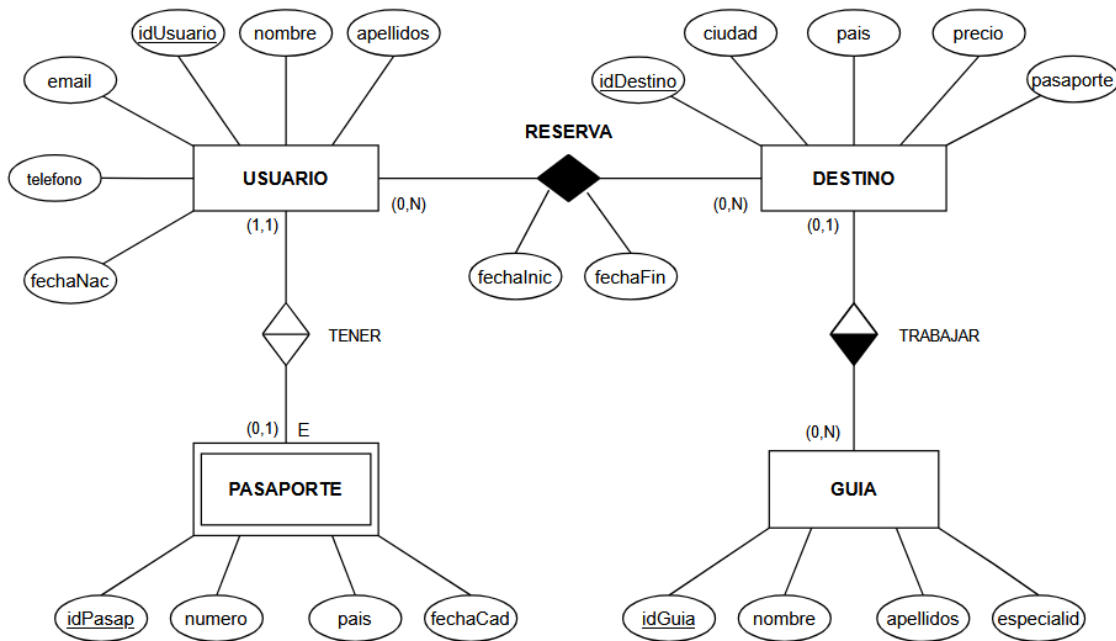
- Usuario
- Destino
- Guía
- Pasaporte

Relaciones:

- Usuario — Destino → M:M
- Destino — Guía → 1:M
- Usuario — Pasaporte → 1:1

Diagrama ER (descripción):

- Un usuario puede elegir varios destinos, y un destino puede ser elegido por varios usuarios.
- Un destino puede tener varios guías, pero un guía solo puede trabajar en un destino.
- Un usuario puede tener como máximo un pasaporte, y un pasaporte pertenece a un único usuario.



4.2 Esquema Relacional

Se crea una nueva tabla, RESERVA, por la relación M:M.

- **USUARIO** (idUsuario, nombre, apellidos, email, teléfono, fechaNacimiento)
 - PK (idUsuario)
- **DESTINO** (idDestino, ciudad, país, precio, pasaporte)
 - PK (idDestino)
- **PASAPORTE** (idPasaporte, numero, país, fecha, idUsuario)
 - PK (idPasaporte)
 - UK (idUsuario) (por relación de Existencia de 1 a 1)
 - FK (idUsuario) → USUARIO (por relación de Existencia de 1 a 1)
- **GUIA** (idGuia, nombre, apellidos, especialidad, idDestino)
 - PK (idGuia)
 - FK (idDestino) → DESTINO (por relación de Muchos a 1)
- **RESERVA** (idUsuario, idDestino, fechaInicio, fechaFinal)
 - (NUEVA TABLA, por relación Muchos a Muchos)
 - PK (idUsuario, IdDestino)
 - FK (idUsuario) → USUARIO
 - FK (idDestino) → DESTINO

Se crean los campos fechaInicio y fechaFinal, por ser necesarias para identificar cada viaje.

4.3 Esquema Físico

El modelo físico implementa:

- Claves primarias
- Claves ajenas
- Restricciones
- Tipos enumerados
- Relaciones 1:1, 1:N y M:M

```
-- archivo creacion de tablas (reserva se crea por relacion M:M usuario-destino)

drop table if exists usuario cascade;
drop table if exists destino cascade;
drop table if exists guia cascade;
drop table if exists pasaporte cascade;
drop table if exists reserva cascade;

-- tabla usuario (se comprueba que sea mayor de edad)

create table usuario (
  id integer generated by default as identity,
  nombre varchar(255),
  apellidos varchar(255),
  email varchar(255),
  telefono varchar(255),
  fecha_nacimiento date,
  primary key (id),
  constraint CK_mayor18 check (fecha_nacimiento <= current_date - interval '18 years')
);

-- tabla destino

create table destino (
  id integer generated by default as identity,
  ciudad varchar(255),
  pais varchar(255),
  precio numeric(10,2),
  requiere_pasaporte boolean,
  primary key (id)
);

-- tabla guia (se crea un tipo, para usar enum)

drop type if exists especialidad_guia cascade;
create type especialidad_guia as enum ('Geografia','Historia','Arquitectura','Comida');
create table guia (
  id integer generated by default as identity,
  nombre varchar(255),
  apellidos varchar(255),
  especialidad especialidad_guia,
  primary key (id),
  id_destino integer unique,
  constraint FK_guia_destino foreign key (id_destino) references destino
);
```

```
-- tabla pasaporte (se comprueba que el pasaporte no esté caducado a dia de hoy)

create table pasaporte (
  id integer generated by default as identity,
  numero varchar(255),
  pais_expedicion varchar(255),
  fecha_caducidad date,
  primary key (id),
  id_usuario integer unique,
  constraint FK_pasaporte_usuario foreign key (id_usuario) references usuario
    on delete restrict
    on update cascade,
  constraint CK_caducidad check (fecha_caducidad > current_date)
);

-- tabla reserva (se añaden campos propios fecha_inicio y fecha_fin)

create table reserva (
  id_usuario integer,
  id_destino integer,
  fecha_inicio date,
  fecha_fin date,
  primary key (id_usuario, id_destino),
  constraint FK_reserva_usuario foreign key (id_usuario) references usuario
    on delete restrict
    on update cascade,
  constraint FK_reserva_destino foreign key (id_destino) references destino
    on delete restrict
    on update cascade
);
```

5. Arquitectura en capas (Java)

La arquitectura sigue el patrón:

Controller → Service → Repository → Entity

Cada capa tiene una responsabilidad clara:

- Entity: representación de las tablas de la base de datos.
- Repository: acceso a datos mediante JPA.
- Service: lógica de negocio.
- Controller: exposición de endpoints REST.
- DTO + Mapper: evita exponer entidades directamente al frontend.

5.1 Entities

Representan las tablas de la base de datos y sus relaciones:

- Usuario

- Destino
- Guía
- Pasaporte
- Reserva

Incluyen anotaciones para relaciones:

- @OneToOne
- @OneToMany
- @ManyToOne
- @ManyToMany

5.2 Repository Layer

Cada entidad tiene un repositorio que extiende JPA:

- UsuarioRepository
- DestinoRepository
- GuiaRepository
- PasaporteRepository
- ReservaRepository

5.3 Service Layer

Define las operaciones CRUD:

- findAll
- findById
- save
- update
- delete

5.4 Service Implementation

Implementa la lógica de negocio:

- Validaciones

- Gestión de relaciones
- Control de errores

5.5 Controller REST

Expone los endpoints:

- GET
- POST
- PUT
- DELETE

Para:

- Usuarios
- Destinos
- Guías
- Pasaportes

6. Frontend (HTML, CSS, JS)

El frontend incluye:

- Formularios para crear entidades
- Tablas para listarlas
- Botones para editar y eliminar
- Llamadas Fetch API al backend

Estructura:

- index.html (y otras html en /pages)
- estilo en /css
- código en /js
- ilustraciones en /img

7. API REST

Usuarios:

- GET /api/usuarios
- GET /api/usuarios/{id}
- POST /api/usuarios
- PUT /api/usuarios/{id}
- DELETE /api/usuarios/{id}

Destinos:

- GET /api/destinos
- POST /api/destinos
- PUT /api/destinos/{id}
- DELETE /api/destinos/{id}

Guías:

- GET /api/guias
- POST /api/guias
- PUT /api/guias/{id}
- DELETE /api/guias/{id}

Pasaportes:

- GET /api/pasaportes
- POST /api/pasaportes
- PUT /api/pasaportes/{id}
- DELETE /api/pasaportes/{id}

Reservas:

- GET /api/reservas
- POST /api/reservas

- PUT /api/reservas/{id}
- DELETE /api/reservas/{id}

8. Metodología GitFlow

Ramas:

- **main**: contiene el código en producción, el código estable y listo para usuarios.
- **develop**: la rama develop se utiliza para integrar los cambios de desarrollo antes de pasar a producción (rama main). En develop se juntan las funcionalidades nuevas, contiene el código en desarrollo que ya funciona, pero aún no está listo para producción
- **feature/***: se ha creado una rama feature por cada miembro del proyecto para que trabajen desde ella. Por tanto tendremos 4 ramas feature: feature/maria, feature/german, feature/cristian y feature/sonia. Estas ramas son donde se desarrollan las nuevas funcionalidades.

Flujo:

feature → develop → main

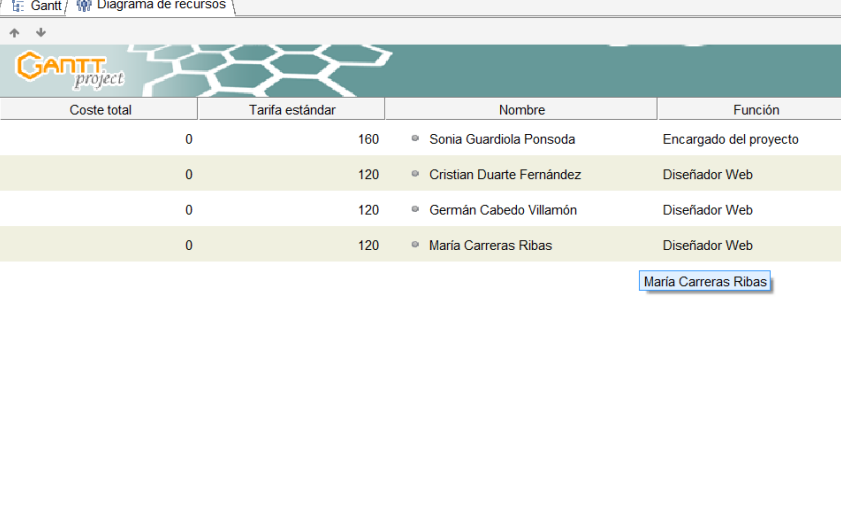
9. Planificación (Diagrama de Gantt)

Fases:

- Diseño/Gestión Base de Datos
- Diseño Web (FrontEnd)
- Desarrollo lógica BackEnd
- Integración FrontEnd - BackEnd
- Pruebas
- Documentación

Capturas iniciales

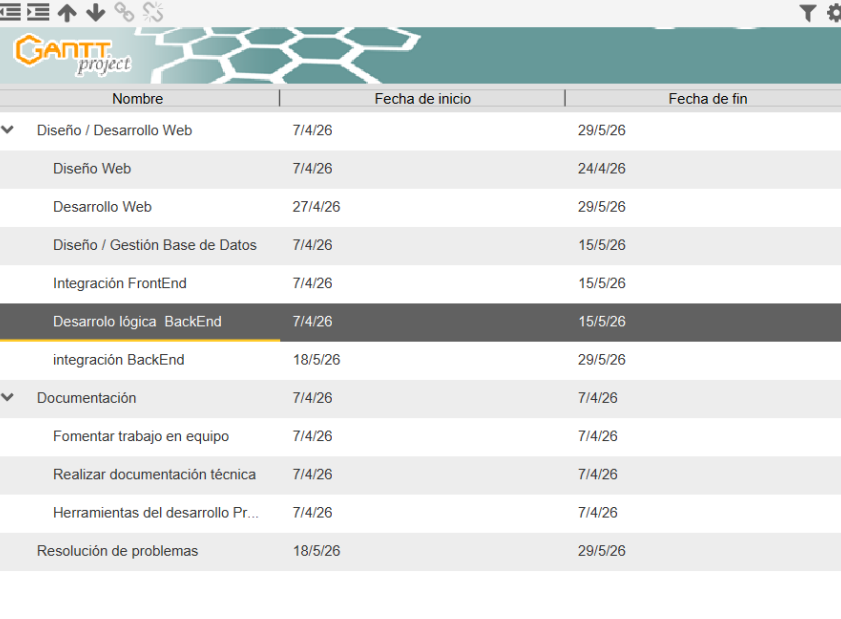
Gantt Diagrama de recursos



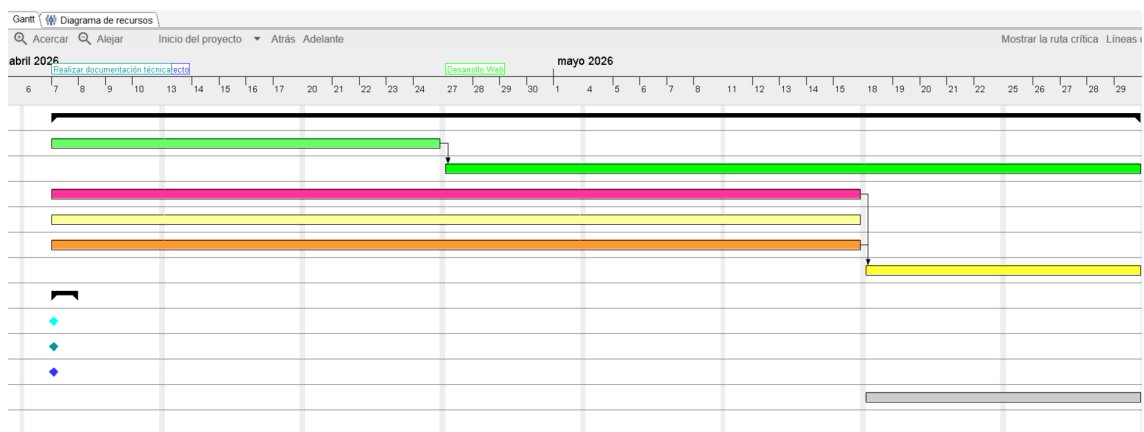
Coste total	Tarifa estándar	Nombre	Función
0	160	• Sonia Guardiola Ponsoda	Encargado del proyecto
0	120	• Cristian Duarte Fernández	Diseñador Web
0	120	• Germán Cabedo Villamón	Diseñador Web
0	120	• María Carreras Ribas	Diseñador Web

Maria Carreras Ribas

Gantt project



Nombre	Fecha de inicio	Fecha de fin
▼ Diseño / Desarrollo Web	7/4/26	29/5/26
Diseño Web	7/4/26	24/4/26
Desarrollo Web	27/4/26	29/5/26
Diseño / Gestión Base de Datos	7/4/26	15/5/26
Integración FrontEnd	7/4/26	15/5/26
Desarrollo lógica BackEnd	7/4/26	15/5/26
integración BackEnd	18/5/26	29/5/26
▼ Documentación	7/4/26	7/4/26
Fomentar trabajo en equipo	7/4/26	7/4/26
Realizar documentación técnica	7/4/26	7/4/26
Herramientas del desarrollo Pr...	7/4/26	7/4/26
Resolución de problemas	18/5/26	29/5/26



Proyecto realizado por (orden alfabético): GERMÁN CABEDO VILLAMÓN, MARÍA CARRERAS RIBAS, CRISTIAN DUARTE FERNÁNDEZ y SONIA GUARDIOLA PONSODA.

Se han realizado algunos cambios en las tareas debido al desconocimiento y la planificación inicial de las mismas. Según hemos ido avanzando en el proyecto, hemos conocido que tareas se realizan primero y cuales después. De ahí los cambios en los periodos de trabajo y los recursos.

Adjuntamos el documento con el resumen del diagrama de Gantt junto con los archivos.

A continuación se adjunta el diagrama Web del proyecto: [DiagramaWebProyecto](#)

10. Conclusiones

El proyecto cumple todos los objetivos planteados:

- Estructura profesional
- CRUD completos
- Conexión frontend-backend
- Base de datos bien diseñada
- Documentación técnica completa
- Flujo GitFlow aplicado

El desarrollo de este proyecto ha permitido aplicar de forma práctica los conocimientos adquiridos durante el curso, integrando distintas tecnologías tanto en el frontend como en el backend para construir una aplicación web completa y funcional.

Se ha logrado implementar una arquitectura sólida basada en el modelo cliente-servidor, con una clara separación de responsabilidades mediante el uso de arquitectura en capas en el backend, lo que favorece el mantenimiento, la escalabilidad y la reutilización del código.

Asimismo, el diseño de la base de datos ha sido clave para garantizar la correcta gestión de la información, estableciendo relaciones adecuadas entre las entidades y asegurando la integridad de los datos.

Por otro lado, la conexión entre el frontend y el backend a través de una API REST ha permitido realizar operaciones CRUD completas de forma eficiente, simulando el funcionamiento real de una aplicación profesional.

El uso de la metodología GitFlow ha facilitado la organización del trabajo en equipo, permitiendo una gestión ordenada del desarrollo y la integración de funcionalidades sin comprometer la estabilidad del proyecto.

En conclusión, el proyecto no solo cumple todos los objetivos planteados, sino que también representa una base sólida para futuros desarrollos en segundo de DAW,

pudiendo ampliarse con nuevas funcionalidades, mejoras en la interfaz o incluso su despliegue en un entorno real.